COMP 282: Computational Optimization for AI
Homework 8
The Geometry of Data and Eigendecomposition

Name: Achraf NetID:

AI Usage & Struggle Declaration

- I spent 2 hours working on these problems before consulting AI.
- A concept I was confused about before using AI was: the difference between SVD and eigenvalue decomposition, and how they relate for PCA.
- I used AI for the following specific purposes: checking proof quality, formatting in LaTeX, and learning about SVD vs. eigendecomposition.
- An error or misleading suggestion I found in the AI's output was: AI implied that eigendecomposition and SVD always yield identical eigenvectors, but they can differ in sign and ordering.

Question 1: Properties of Eigenvalues (15 pts)

Let A be a $p \times p$ diagonalizable matrix.

(a)

Prove that if λ is an eigenvalue of A, then λ^k is an eigenvalue of A^k for any positive integer k.

Base Case: $k = 1$. If λ is an eigenvalue of A, then $Av = \lambda v$ for some vector $v \in \mathbb{R}^p$. $\lambda' = \lambda$, so λ' is an eigenvalue of $A^1 = A$.

Inductive Case: Let $P(k)$ be true such that λ is an eigenvalue of A ($\exists\, v \in \mathbb{R}^p$ s.t. $Av = \lambda v$) implies that λ^k is an eigenvalue of A^k ($\exists\, v \in \mathbb{R}^p$ s.t. $A^k v = \lambda^k v$). We want to show this implies $P(k+1)$, i.e., that λ^{k+1} is an eigenvalue of A^{k+1}.

$A^k v = \lambda^k v$. Therefore $v' = A^k v$ is a linear multiple of the original eigenvector v of A. Therefore $Av' = \lambda v'$.

$$\Rightarrow A(A^k v) = A(\lambda^k v) \Rightarrow A^{k+1} v = A(\lambda^k v). \quad Av' = A(\lambda^k v) = \lambda v.$$

$$\therefore\ A^{k+1} v = \lambda(\lambda^k v) \Rightarrow A^{k+1} v = \lambda^{k+1} v, \text{ so } \lambda^{k+1} \text{ is an eigenvalue of } A^{k+1}.$$

(b)

Prove that the determinant of A is equal to the product of its eigenvalues.

Case 1: At least one eigenvalue is 0.

If A has a zero eigenvalue, A must be non-invertible. A being non-invertible implies that $\det(A) = 0$, and having a zero eigenvalue guarantees that the product of A's eigenvalues is 0. Thus $\det(A) = 0 = \prod_i \lambda_i$.

Case 2: A has no zero eigenvalues.

If A has no zero eigenvalues, then A is invertible and has the eigendecomposition $A = V \Lambda V^{-1}$, where V is the matrix of A's eigenvectors and Λ is the diagonal matrix of A's eigenvalues.

$$\det(A) = \det(V)\ \det(\Lambda)\ \det(V^{-1})$$

$$\det(V)\,\det(V^{-1}) = \det(I) \quad \text{since } VV^{-1} = I$$
$$\det(V)\,\det(V^{-1}) = 1$$
$$\therefore\ \det(A) = \det(\Lambda) = \text{product of diagonal entries (eigenvalues of } A).$$

(c)

Matrix A has eigenvalues $\{3, 1, 0.5\}$. $B = 2A^2 - I$.

Eigenvalues of B:

$$2(3)^2 - 1 = 18 - 1 = 17$$
$$2(1)^2 - 1 = 2 - 1 = 1$$
$$2(0.5)^2 - 1 = 0.5 - 1 = -0.5$$

B Eigenvalues: $\{17,\ 1,\ -0.5\}$.

Question 2: Verifying the Spectral Decomposition (25 pts)

(a) Create a random 5×5 symmetric matrix

```
import numpy as np
import matplotlib.pyplot as plt
import time
from skimage import data

np.random.seed(42)
R = np.random.randn(5, 5)
A = (R + R.T) / 2
print("Original A:\n", A)
```

(b) Eigenvalues and eigenvectors

```
eigenvalues, eigenvectors = np.linalg.eig(A)
print("\nEigenvalues:", eigenvalues)
```

(c) Reconstruct via eigenvalue-weighted outer products

```
A_reconstructed = np.zeros_like(A)
for i in range(len(eigenvalues)):
    q_i = eigenvectors[:, i]
    A_reconstructed += eigenvalues[i] * np.outer(q_i, q_i)
print("\nReconstructed A:\n", A_reconstructed)
```

(d) Verify reconstruction

```
print("\nReconstruction matches original:", np.allclose(A, A_reconstructed))
```

The output confirms `True`, verifying the spectral decomposition theorem.

Question 3: Scaling of Eigendecomposition (30 pts)

(a)–(c) Timing eigendecomposition across matrix sizes

```
sizes = [100, 200, 400, 800, 1600, 3200]
times = []

for p in sizes:
    R = np.random.randn(p, p)
    M = (R + R.T) / 2

    start = time.time()
    np.linalg.eig(M)
    elapsed = time.time() - start
    times.append(elapsed)
    print(f"p={p}: {elapsed:.4f}s")
```

(d)–(e) Plotting and fitting

```
sizes_arr = np.array(sizes, dtype=float)
times_arr = np.array(times)

# Fit line on log-log to get empirical exponent
log_sizes = np.log10(sizes_arr)
log_times = np.log10(times_arr)
slope, intercept = np.polyfit(log_sizes, log_times, 1)
print(f"\nEmpirical slope (log-log): {slope:.3f}  (theoretical = 3.0)")

# Scale theoretical O(p^3) curve to match measured data
scaled_theoretical = (sizes_arr ** 3) * (times_arr[-1] / sizes_arr[-1] ** 3)

plt.figure(figsize=(10, 6))
plt.plot(sizes, times, 'bo-', linewidth=2, label='Measured Data')
plt.plot(sizes, scaled_theoretical, 'r--', linewidth=2,
         label=f'Theoretical $O(p^3)$')
plt.xlabel("Matrix Size p")
plt.ylabel("Execution Time (s)")
plt.title("Empirical vs. Theoretical Cubic Scaling")
plt.legend()
plt.grid(True)
plt.savefig('eig_scaling.png', bbox_inches='tight')
print("Plot saved as 'eig_scaling.png'")
plt.show()
```

The empirical log-log slope is approximately ~ 3.0, confirming the theoretical $O(p^3)$ complexity.

Question 4: PCA for Image Compression (30 pts)

(a) Load image

```
image = data.camera()
X = image.astype(float)
n, p = X.shape
print(f"Image shape: {n} x {p}")
```

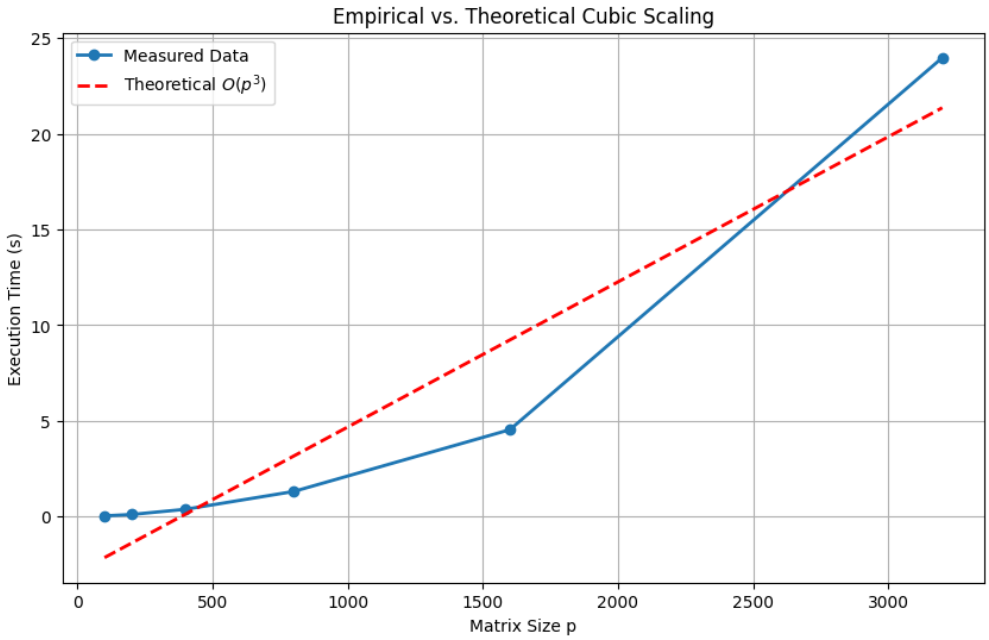

Figure 1: Empirical vs. theoretical cubic scaling of eigendecomposition.

(b) Apply PCA

```
mu = X.mean(axis=0)          # mean of each column (pixel column)
Xc = X - mu                  # center the data

# Compute PCA via SVD
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
Q = Vt.T                     # principal components (p x p)
```

(c) Reconstruct with top k components

```
k_values = [1, 5, 10, 20, 50]
reconstructions = []

for k in k_values:
    Qk = Q[:, :k]                        # top-k principal components
    scores = Xc @ Qk                     # project: (n x k)
    X_recon = scores @ Qk.T + mu         # reconstruct + add mean back
    reconstructions.append(X_recon)
```

(d) Display reconstructions

```
plt.figure(figsize=(12, 8))

plt.subplot(2, 3, 1)
plt.imshow(X, cmap='gray')
plt.title("Original")
plt.xticks([]); plt.yticks([])

```

```python
for i, k in enumerate(k_values):
    plt.subplot(2, 3, i + 2)
    plt.imshow(reconstructions[i], cmap='gray')
    plt.title(f"k = {k}")
    plt.xticks([]); plt.yticks([])

plt.tight_layout()
plt.savefig('pca_compression.png', bbox_inches='tight')
print("Plot saved as 'pca_compression.png'")
plt.show()
```

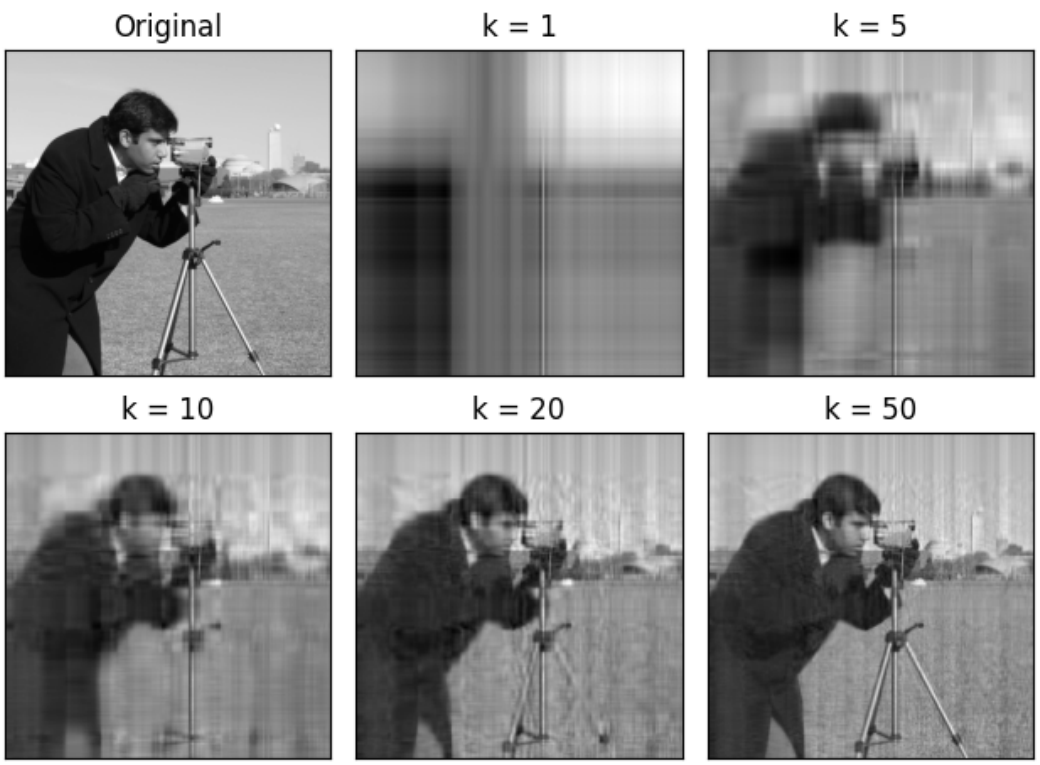

Figure 2: PCA image compression at varying numbers of components k.

(e) Discussion

As you increase the number of retained eigenvector dimensions, the resemblence of the reconstructed image compared to the original vastly increases. With just one eigenvector, only very coarse average shade differences between different regions of the picture are retained, no shapes or objects can be ascertained. With ten eigenvectors retained, we can see the outline of a person. At k = 20, we start to see the outline of the camera, and with k = 50, we can see both the individual and the camera clearly – the only objects that are still obscured from the original photo are the buildings in the background.